Summarize

Chapter 14. Monitoring and Observability Patterns

One of the core differences between client applications and distributed systems is that generally distributed systems implement services. Services are always on, always available for users around the world in all time zones and ways of working. Because of the 24/7 nature of these systems, monitoring and observability become critical to building reliable systems. To deliver reliability, you must notice a problem before the customer notices a problem; and to solve any problems you find, you need to be able to understand how your system is operating. This chapter focuses on best practices for such monitoring and observability.

Monitoring and Observability Basics

Before we get into the details of implementing monitoring and observability, it is useful to ground ourselves in the core set of concepts that make up any monitoring and observability solution.

In any system, there are four key concepts which make up our solutions:

- Logging
- Metrics
- Alerting
- Tracing

We'll step through each of these in a little more detail.

It's highly likely that anyone who has built even the smallest system has implemented logging, even if they don't realize that they have. The simplest version of logging is the humble `printf` statement. Of course, there are many more sophisticated ways to do logging, but ultimately they all serve the same purpose as that print statement. Namely, they show us that a particular place in our code has executed, and they record data asso-

ciated with that place in the code. Logging records data that helps us understand specific executions of our code.

The contrast to logging are metrics or monitoring. While logging is associated with a specific execution of our source code, a metric is generally related to aggregate data across multiple requests. Examples of metrics could be the number of times this particular function has been called in the last minute, or the average length of time spent in that function across all of these calls. While logging provides us a perspective to understand a specific execution of our service, metrics provide us a global perspective of how our service is executing in general.

The purpose of logging and metrics are to help us understand our systems when we are looking at them, or when we know that there is a problem. If we spent all of our time staring at graphs of metrics and logging interfaces, that might be all we need; but of course, in real life we spend much of our time in other environments. Consequently, we need *alerting* to detect problems and notify us that we need to interrupt our lives and go look at the logging or metrics. Effectively, alerts are rules that are applied to either logs or metrics which trigger an event. Generally, this event triggers a notification which is intended to draw our attention to a problem (or potential problem) in our systems. Crafting high-quality alerts is a key part of building a reliable system.

The final component in our system relates to the distributed part of our distributed system. One of the hardest problems in building out a distributed system is understanding how all of the pieces work together, or even harder, understanding why they may fail in certain circumstances. From a naive point of view, every individual request to a microservice is unique and individual. But the truth is that from the broader perspective of the application, there is often a larger request, which is being serviced by a chain of calls through all of the microservices. *Tracing* is the act of rebuilding this context so that instead of seeing individual, unique requests at each microservice, we can understand that all of these individual pieces are part of a broader user request. This global perspective, in turn, enables us to understand (for example) why a particular user's request is running slowly or failing to work at all. Tracing takes all of the pieces of a distributed system and reconstructs the user's perspective.

In the coming sections, we'll dive deeper into each of these components and see how you can use them to build a reliable system.

Logging

Once you figure out how to log information, one of the first things that you notice in a distributed system is just how much information gets logged. The volume of logs, or "log spam" can often be a significant problem in terms of understanding a problem, or even just the raw cost of storing and searching all of those logs.

The truth is that the value of logging is very contextual. When you are debugging a specific problem in a specific component, then verbose, detailed logging can be critical to identifying the problem. On the other hand, during standard execution, where everything is working correctly, most of the logging is just wasted noise, except those logs which trigger alerts because of system errors.

Consequently, only the simplest distributed systems use logging like basic printing. Nearly every system uses some sort of logging library. These logging libraries provide important capabilities that make it much easier to deal with lots and lots of logs. The most basic capability provided by such libraries is basic timestamping. Oftentimes we are trying to understand what happened and in what sequence; and without a shared notion of time in every log line, it is difficult to know (for example) the amount of time that passed between different logs. In addition to timestamp, most logging libraries enable the addition of other types of context to the logs. In any multithreaded system, many requests may be processed in parallel, and thus many different identical logs may be issued. To differentiate between these logs and understand how they relate to a specific request, context like the thread identifier, user ID, or request ID help differentiate between otherwise identical logs on the same server and filter to the context of a specific request. The final, and often most valuable, service provided by logging libraries is leveled logging. Leveled logs allow you to express the type of logs being emitted. The specific number of different levels often vary by library, but in general they include:

Debug

: Verbose logs, generally only useful when doing detailed debugging.

Info

: Often useful but not exceptional messages tied to the operation of the service.

Warning

: Something that is concerning, but not necessarily fatal to the entire application.

Error

Something very bad has happened, and someone should take a look as soon as possible.

Fatal

Something unrecoverable has happened, and the application will terminate after writing this log.

Leveled logging enables you to filter only to certain types of messages, and it allows you to leave detailed logging in your code, for when it is needed, without filling up your log streams with many, many spammy log messages. Generally, your log level for an application is set to Info or Warning depending on your logging style. Debug messages are reserved for conditional activation to debug specific problems.

In addition to the request context, a valuable context for the logging library is the location of the log call itself. Typically, logging systems understand which class or component contains the log. This enables *conditionally logging* where you dynamically activate detailed debug logging for specific classes or other components on running servers. Conditional logging enables you to respond to a reported incident or a problem seen by a specific user without requiring the storage or latency of detailed logging for every request. Some care should be used when performing conditional logging, as it can significantly increase the latency of end-user requests. Nothing is worse than causing a worse incident or outage in the process of investigating an outage.

Of course, to make logging useful, you must have a clear sense of what the purpose of logging is in the first place. Ultimately, logging provides insight into the operation of your service or application. One common joke is that the way to know what to log is to remember what you were missing when you initially debugged the problem. That is, logging is often a retrospective exercise "I wish I had logged...." It goes without saying that the most useful things to log are exceptional or erroneous situations. Every error or exception should probably be logged unless you are completely confident in the error-handling code (and sometimes even then). Beyond errors, often the most interesting things to log are the things that are weird. For example, some requests are just slow. You can stare at metrics (see "Metrics") showing that your 99th-percentile requests take a ridiculous amount of time, but without logs indicating which parts of request handling are exceptionally slow, it is hard to take action. On the other hand, just logging the play by play ("function foo was called") is rarely going to be that useful, and it is definitely going to spam your logs. When logging, try to imagine the hardest problem you might need to debug in that particular part of the code, and write down in the log whatever you would need to successfully figure it out.

Metrics

In contrast to logging, with its focus on individual executions, monitoring metrics are designed to collect statistics over time which characterize the general execution of your services. The focus of this metrics section will be the Prometheus monitoring application, which has become the de facto industry standard. Other monitoring solutions offer similar capabilities.

Monitoring data is generally recorded over time; from this view you can see both the instantaneous state of your application and also how that state is changing over time. When monitoring your application there are three basic types of data that are interesting to record:

- Histograms
- Counts
- Values

Histograms represent a distribution of values over some interval. Histograms enable you to understand both the average and the extreme experiences of your users. For example, a histogram of latency can show you the best 10% average and worst 10% latency that your users experienced. Such information is useful to give you an "at-a-glance" sense of how your application is performing.

Counts are monotonically increasing values that keep track of some value that can only increase. One of the most common examples of a count is the total number of requests served by your service. This number can clearly only increase over time, as there are more and more requests. Counts are also often transformed into *rates*, which are the first-order derivative of the count and represent the rate of change of that count over time. Given a count that tracks the total number of requests, the rate could give you the number of requests per second. Counts are inherently integral (whole numbers) in nature.

Finally, there are values; these are numbers which can take on any value and can both increase and decrease over time. Common examples of values include CPU and memory usage for a service. Clearly, the amount of CPU and memory use can vary over time and increase or decrease based on the operation of the service. Values are most useful when viewed over time because they can indicate the overall behavior of the system. Steadily increasing memory usage can indicate a memory leak. Periodically spikey CPU can indicate some background process which is unnecessarily expensive.

It should be clear at this point that the main value of monitoring data is the perspective that it gives over time. Indeed, most monitoring is referred to as a *time series* representing exactly these values over time. Dedicated databases for this time series data have been developed to efficiently store and query this data. The Prometheus system comes with a simple time-series database, but most people will integrate Prometheus with a dedicated time series database. Often, a good option is to integrate Prometheus monitoring with a cloud-based monitoring as a service. At this point, most cloud-based monitoring services, for example Azure Monitor, support the ingestion of Prometheus metrics.

When it comes to ingesting metrics, Prometheus is a pull-based or scraping metrics system. Periodically the Prometheus application sends requests out to your application for the current state of all of the metrics. These metrics are stored into the time-series database for later query and recording. Pull-based monitoring works well for applications that are always running, but how can you record metrics for batch and other transient jobs that may not be around long enough to be scraped? For such jobs, you need to implement *push-based* metrics instead. Fortunately, Prometheus has a sister project, the Prometheus push gateway, which implements exactly this pattern. When you run the push gateway, it runs a dedicated server that Prometheus can scrape. Your applications can send or "push" metrics into this gateway so that they are recorded even if your application is no longer running. Combining pull-based monitoring for your long-running applications and push-based metrics for transient jobs allows you to have a complete perspective on the operation of your application.

Basic Request Monitoring

One of the first types of monitoring that most people add is for the number of requests served by your application. This is a basic statistic that demonstrates how many people (or other services) are using your service over time. The number of requests is an example of a count metric. The number of requests cannot decrease since there is no such thing as a negative request.

However, the total number of requests is actually not that interesting, what is more interesting is the rate of change of the total number of requests. Rate of change is a secondary metric defined as the difference between the number of requests at time *t* and the number of requests at time *t + 1*. To make this concrete, the rate of requests per minute is the total number of requests right now, minus the total number of requests one minute ago. To turn this into a graph, the rate is calculated for each previous minute moving backward and producing a rate at one-minute intervals. Of course, in this exam-

ple, one minute is arbitrary; you can do equivalent calculations for whatever time interval that you want, from a millisecond to a day (or more).

Thinking about calculating the rate of requests for a millisecond demonstrates one of the challenges with rate metrics. If your server serves fewer than 1,000 requests per second, calculating the rate of requests at millisecond granularity will lead to a very spikey metric, which oscillates between zero and one (and possibly more if there is a brief surge of requests). Rates are variable and highly dependent on external factors. Consequently, it is often more useful to use the average of the rate of requests per time interval, rather than just the raw value. Averaging the rate over time smooths the peaks and valleys and gives you a more realistic picture of how your service is being accessed.

The total number of requests is a good metric to start with, but to truly understand the operation of your system, it is useful to look at more details associated with each request. One of the most common features of a request is the HTTP response code which it returned to the users. If you are not already familiar with HTTP response codes, they are a part of the HTTP protocol that indicates how the server responded to the request. Some of the most common are 200 (OK), 404 (Not found), and 500 (Internal Server Error), but there are many more response codes for different situations. With Prometheus metrics, values can have labels which add detail to the metric. Using this capability, we can view the response code as a label associated with the request count. Using this label, we can query for the total number of requests which were served successfully (`code==200`) or the total number of requests which resulted in errors (`code==500`); we can also calculate the rate of each of these types of requests using the methods described previously. Subdividing the total request count by the response code gives us a much richer view into the operation of our application. We can also group error codes together and view how many requests were errors (codes starting with 5xx) and how many requests were successful (codes starting with 2xx).

In addition to the response code for a server request, the next most important characteristic of the request is the latency of the request, or how much time it took to process. It might be tempting to think that latency could be a label on the request count as well, but this doesn't work. Labels need to be a limited set of discrete values, while latency is a value that can take on many different arbitrary values. Request latency is a separate metric, and it is generally modeled as a histogram. Using the histogram metric for request latency, you can understand the average experience of the user as well as the experience of users who have the slowest requests. As with request count, the request latency can also be labeled using the response code. That way, not only can you see the average experience of any request, but you can also calculate the average latency of an error, of a not found request, or any other HTTP response code.

Advanced Request Monitoring

Though total request count, the rate of requests, and latency are the most common and most generally useful metrics for monitoring requests to your server or service, there are other more specialized metrics, which can be quite useful to have a deep understanding of how your service is performing. A good example of this might be the request and response sizes in bytes for your service. Request size is useful because you can use it to correlate to slow requests (larger requests may take longer to parse and process) and also to look at it over time to detect patterns like increasing requests or response sizes over time. Like latency, request and response size are examples of histogram metrics.

Sometimes useful metrics are recorded by adding detail to how you calculate and report metrics in your application. For example, it is sometimes useful to understand the amount of time a request has spent in a queue, versus how much time was spent processing a request. This sort of information is especially useful when you are debugging why your service is slow in processing requests. Simply seeing that latency is high will show you that there is a problem and that users are likely unhappy, but having metrics for queue time and processing time will help you understand whether the problem is that you need to scale out (queue time is too long) or if you need to optimize your code (processing time is too long). To get these metrics, you actually need to add additional metrics to your code. In this case, you would add two new histogram metrics, one measuring the time from when a request is received to when processing starts, and one measuring the time from when processing starts to when the request is completed. This system now has three histogram metrics measuring latency: total latency, queue latency, and processing latency. Metrics are cheap to add, and the additional insight that they provide is often invaluable in debugging your service. Of course, as with anything, it is very possible to overdo things, and if you are monitoring the latency of each line of code (or even each function), you're probably taking things a little too far.

Another advanced metric for requests might be to break them down by geography or customer. This monitoring can give you business insights that can help you build better products, or possibly identify abusive requests coming from specific users or locations. If you are willing to only look at coarse-grained geography (e.g., country), then it is possible to view geography as another label that you can apply to the request count metric; but more often, this sort of data needs to be obtained by querying across logs instead of looking at metrics directly. For example, in any large system there are far too many customers to use a customer ID as a label for the request count metric. Instead, the system logs the customer ID whenever a request is processed. This example demonstrates the interplay between logging and monitoring. Monitoring and metrics can provide aggregate data that is quick to query. For detailed information, logging and log search is an

essential technique that provides slower, but more rich, access to data. In this case, because the business analysis can likely stand extra latency in processing, obtaining customer usage information from log search can meet the business requirements.

Alerting

So far we've discussed all of the different ways to record information from your application's logs and metrics. This data can be critical for understanding why your system isn't behaving correctly during an outage or other incident. But how can you figure out something is happening in the first place?

Obviously, the first way we find out that there is a problem in our system is when a customer or user complains. But waiting for such *customer reported incidents*, or CRIs, is a bad experience for both the user and the person receiving the alert. Nothing hurts customer confidence like being told that they were the first one to notice a problem. At this point with our systems, every user expects that we will notice (and fix) problems in our services before the user notices that anything is amiss.

Thus, the best way to find out about a problem is via *alerting*. Alerting is the practice of establishing conditions under which the team responsible for a service is notified, typically through a pager alert, that there is a problem with the system.

Basic alerting

The most basic form of alerting is based on metric queries and static thresholds. For example, you might choose to alert if the latency at the 90th percentile in your system is greater than half a second. You might also chose to alert if the number of HTTP 500 errors reported by your system is greater than 1% of all requests. Given these rules, it's easy to see that if either occurred it would be a good idea to let the engineers know; but how do you go about determining the right alerts for your system?

The most basic way to think about writing alerts is that they define the service level objective (SLO) for your application. If you don't alert, you won't notice and so your alert thresholds define the difference between nominal and exceptional operation. Every application is different, but it is good to write down what the goals are for a "normal" user experience of your service and right alerts that fire if those goals aren't being achieved. Another consideration is the experience of your on-call engineers. Firing alerts constantly is a recipe for burnout and people quitting. Similarly, firing alerts that are false alarms ("crying wolf") will cause your on-call engineers to stop responding. Therefore, you need to balance the goals of your service with the needs of your on-call engineers to

determine the right alerts. The development of alerts is an ongoing activity throughout the lifespan of your service. As you add more capabilities and, thus, more monitoring, you should also be adding more alerting; as you get more experience with the system, you can tune the alerts to be tighter or looser depending on the needs of your users or your on-call engineers.

Alerting on anomolies

Basic alerting is great for many use cases, but sometimes it is insufficient. A great example of this is an error that causes a 100% outage for a small (1%) subset of your users. From a static alerting perspective, your system is functioning within bounds, but for those poor users in that subset your service is completely down. Static alerting assumes that all metrics are uniform across all users and service calls. However, this isn't always the case. Anomoly detection is a form of AI that can identify anmolies: for example, customer A is 100% down while customer B is working perfectly. Though alerting on anomolies is an advanced topic, it is often the only way to find a balance between alerting when there is a problem and firing lots of false alarms. As your system gets larger and more complex, anomoly detection alerting will become a critical requirement.

Monitoring and logging provides visibility into your application. Alerting tells you when you should be paying attention. Importantly, not every metric requires an alert; some are useful only for debugging and visualization. However, the performance of your application will be defined by the quality of your alerts. Invest in alerting with the same energy you bring to designing customer-facing features.

Tracing

As we have seen in the preceding chapters, most modern applications involve requests that span multiple machines and processes. From the techniques above, you can learn how to monitor all of these components individually. But unfortunately, this service-by-service view shows you all of the different requests flowing through your system at the same time, but it doesn't do a good job showing you the path of a single request through all the microservices in your system. Request tracing correlates all of the service metrics together and gives you that end-to-end perspective.

The easiest way to achieve tracing is to create a unique correlation ID for every request as it enters your system. Effectively, this correlation ID is a number that can be used to group together logs, metrics, and other information across your application. The correlation ID can be anything you want as long as it uniquely identifies each request. The simplest correlation ID would be a large random number; a better one would be a hash func-

tion applied to unique characteristics of the original request like its RESTful path and source IP address.

Once you have a correlation ID, whenever you emit a log or a metric, you emit the correlation ID. For example, in a logging system, in addition to the timestamp and the log level, you would also log the request correlation ID. To enable you to log that ID within all components in your system, you need to pass it along with the API calls you make throughout your system.

Assuming that you are making API requests using the HTTP protocol, you can embed your correlation ID in an application-specific header, for example, `x-my-apps-correlation-id`, and modify your code to read from that header and populate a request context object with that ID. Once you have correlation IDs populated throughout your logging and metrics, you can group that information back together to visualize the path of a single request through your entire system.

If this sounds like a lot of work to implement and get right, you're right, it is. Fortunately, the OpenTelemetry project has taken care of much of the heavy lifting for you. OpenTelemetry has SDKs for all popular languages that make it easy to integrate request tracing into your application. It has a vendor-agnostic philosophy that allows it to integrate with numerous backends for metrics and logging. Additionally, numerous visualization and debugging tools have also integrated with OpenTelemetry to provide rich introspection capabilities. In nearly all cases, integrating with a community-driven open source project like OpenTelemetry for request tracing is a better idea than building your own.

Aggregating Information

Once you start monitoring, you will discover that monitoring can generate a tremendous amount of information. Efficiently maintaining necessary information as well as being able to quickly sift through it to find the right information for your current issue becomes an important challenge. Aggregation enables you to group information together so that it is easier to understand or more efficient to store.

The first form of aggregation that most people begin with is querying logs across multiple processes. In "Tracing" we discussed how OpenTelemetry provides a request-oriented view of your data, but that's not the only aggregation you might want to perform. If you have a component that has been replicated for scale, you may want to search for instances of a specific error or failure across all of the replicas.

Fortunately, there are numerous tools that make it easier to group logs across multiple containers in Kubernetes. One of the easiest to use is a tool like `ktail`, which extends the traditional `kubectl` log interface to combine logs across multiple pods. More sophisticated tools ingest logs into a search index like Elasticsearch, which makes it possible to run arbitrary queries over your logs, just like you would with web search.

Setting and maintaining a log index is complicated and error-prone, but fortunately, at this point all public clouds and many startups offer log ingestion and search as a service. Similar services are also available for time series metrics. Whether you are building in the public cloud or your own infrastructure, using a service for something as critical as your logs is the right choice more often than not.

In addition to storing your logs and metrics in systems designed for efficient storage and querying, sometimes it is necessary to reduce the information being stored. This can be referred to as downsampling or aggregation. The simplest form of downsampling for metrics is to just store the metric less often. For example, if you are recording the metric every second, after a week you might reduce that to once per minute. Even this simple change reduces the space necessary by 60 times. Of course, simply dropping the sample rate can cause you to lose information. Another option would be to use the average value over those sixty seconds instead of an arbitrary instantaneous value. This simple kind of downsampling can be used for logs. The simplest approach is to keep only a certain class of logs, such as errors, while dropping less important information. You can also just keep a fraction of the logs, though this can be very lossy in terms of information. Finally, with more effort, you can develop application-specific aggregations that maintain the information that is relevant to your application while compressing the data used to store it.

Though downsampling is often a good solution, if you want to reduce costs while maintaining full data fidelity, perhaps for compliance or security, you can instead change the accessibility of the data. Uploading logs to cloud-based "cold" storage can significantly reduce the costs of storing the data, while making it slower and harder to query information from those logs. This can often be a good solution when data needs to be retained for long periods of time, but is infrequently used. Some logging as a service solutions will implement this sort of data tiering for you automatically.

Summary

Monitoring is a critical part of any distributed system. Though it doesn't contribute to making the system operate correctly, when the system is not operating correctly, it is

the key component which enables us to understand what is failing and how to correct it. Learning how to correctly monitor and observe your systems is critical to ensuring that you can rapidly restore reliable operation in the face of a problem or outage.

Previous chapter
< V. Universal Concepts

Next chapter
15. AI Inference and Serving >